The Ultimate Archive Format Comparison: A Reproducible Benchmark of Forty-Two Compression Formats Spanning Four Decades

true

2026-07-09

Abstract

Casual computer users face a bewildering choice of archive formats, from the ubiquitous ZIP to a long tail of modern, vintage, and obscure alternatives. We present the most comprehensive practical comparison to date: a single, fully reproducible benchmark of **42 archive and compression formats**, evaluated on a realistic 55 MB corpus of text, office documents, images, and video that mirrors a typical user's folder. Formats range from the 1984 Unix `compress` (.Z) and the 1985 BBS-era ARC through today's Zstandard, Brotli, and the context-mixing paq8px, and include macOS-native (Apple Archive, compressed DMG), filesystem/imaging (SquashFS, WIM), and formats we resurrect under emulation, including PKZIP 2.04g (1993) run under DOSBox. Every tool is exercised at its default and maximum compression settings; every archive is round-trip verified by SHA-256 against the original bytes; and all raw results and inputs are committed so any reader can reproduce the study with a single command. We report size, speed, and memory per content category, chart the size/speed Pareto frontier, and translate the findings into concrete guidance for everyday users. We also document an empirical format-longevity result: some Windows-only archivers can no longer be run to completion on a modern Apple Silicon Mac even under emulation.

1 Introduction

Compressing a folder of files is one of the most common things a computer user ever does, and yet the choice of *how* to do it has never been more confusing. The graphical "compress" command on Windows and macOS produces a ZIP file whose core algorithm, DEFLATE, was standardized in 1996 [1] and whose lineage runs back to the Lempel–Ziv work of 1977 [2]. Meanwhile, a casual user who looks a little further finds 7-Zip, RAR, and a growing list of modern codecs—Zstandard [3], Brotli [4], and others—each promising smaller files, faster operation, or both. Beyond these lies a long tail of formats that were once dominant and are now curiosities: ARC, ARJ, LHA, zoo, and the venerable Unix `compress`.

The practical question this paper answers is deliberately narrow: **for a typical person compressing a typical folder, which format produces the smallest file while remaining practical to create and to open?** "Practical" is the operative word. We restrict every tool to settings a non-expert could plausibly use—its out-of-the-box defaults, and the single "maximum compression" dial that most archivers expose—and we deliberately avoid the method- and dictionary-tuning that dominates specialist compression benchmarks. Where prior public benchmarks such as the Squash Compression Benchmark [5], lzbench [6], and Matt Mahoney's Large Text Compression

Benchmark [7] measure *codecs* on standardized corpora like Silesia [8], we measure *end-user archive formats*—complete with their container overhead, their real command-line tools, and their round-trip integrity—on a realistic mixed corpus that resembles a home user's folder.

Our contribution is threefold:

1. **Breadth.** We benchmark **42 archive and compression formats** in a single, uniform harness, spanning four decades: from 1984's Unix `compress` (.Z) [9] and 1985's ARC [10] to today's context-mixing paq8px [11]. The set includes mainstream formats (ZIP, 7z, RAR, gzip), modern codecs (Zstandard, Brotli, xz, LZ4, bzip3, lrzip, zpaq), vintage formats (ARC, ARJ, LHA, zoo, HA, freeze, compress), macOS-native formats (Apple Archive, compressed DMG, xar), filesystem/imaging containers (SquashFS, WIM, ISO, cab, dar), and formats we resurrect under emulation—most notably PKZIP 2.04g (1993) run under DOSBox.
2. **Rigor and reproducibility.** Every archive is round-trip verified: it is extracted—often with an *independent* implementation—and compared byte-for- byte against the original via SHA-256. The entire study runs from a single command on committed, public-domain inputs, and every raw measurement is committed as machine-readable JSON so that each number in this paper is auditable and re-derivable.
3. **A longevity finding.** In the course of the study we document an empirical result about format longevity: several Windows-only archivers can no longer be run to completion on a current Apple Silicon Mac, even under the Wine compatibility layer, because they deadlock in their compression cores under x86-to-ARM binary translation.

2 Related work

Codec benchmarks. The most rigorous public compression comparisons focus on raw codecs. The Squash Compression Benchmark [5] wraps dozens of compression libraries behind a single abstraction layer and measures them across machines and datasets; lzbench [6] does likewise in-memory for the LZ-family. Mahoney's Large Text Compression Benchmark [7] and the associated PAQ line of work [12] chart the extreme frontier of ratio on a single large text file. These efforts are authoritative for *algorithm* selection but deliberately abstract away the archive *container*, the actual end-user tool, and file-system realities such as directory structure and mixed content.

Standard corpora. Benchmarks conventionally use fixed corpora—Silesia [8], the Canterbury corpus, or `enwik8`/`enwik9` for text. These enable cross-study comparison but are unrepresentative of a consumer's folder, which mixes already-compressed media (JPEG, H.264) with highly compressible text and semi-structured office documents.

Format references. The formats themselves are documented across standards and primary sources: DEFLATE and gzip [1], [13], the ZIP container [14], Brotli [4], Zstandard [3], the Burrows–Wheeler transform underlying bzip2 [15], LZW underlying `compress` and GIF [9], and the context-mixing/PAQ family [12]. Our work sits between the codec benchmarks and the format references: we ask what an ordinary user actually gets, today, from each *format's real tool*, on *their kind of data*.

3 Methodology

3.1 Corpus

The benchmark corpus is a 55 MB collection assembled to mirror a typical user's "Documents" folder, drawn entirely from public-domain or freely reusable sources so that it can be committed to the repository and redistributed (Table 1). It is partitioned into four categories, each of which is also benchmarked on its own so that we can report where format choice matters:

Table 1: Corpus composition. All files are public domain; provenance and SHA-256 checksums of every source are recorded in `corpus/SOURCES.md`.

Category	Size	Contents
Text	~11 MB	Complete works of Shakespeare and *War and Peace* [16]; a USGS earthquake catalog CSV [17]; a synthetic web-server log
Documents	~15 MB	Generated `.docx`, `.epub`, `.pdf`, `.pptx`, `.odt`, and `.xlsx` files built from the public-domain texts and data
Images	~16 MB	NASA/JWST and Apollo photographs (JPEG, PNG) [18]; uncompressed TIFF and BMP derivatives
Video	~13 MB	Public-domain NASA Apollo footage (H.264 in MP4 and MOV containers) [18]

The mix is deliberate. Office documents (`.docx`, `.xlsx`, `.pptx`, `.epub`, `.odt`) are themselves ZIP containers, so they stress an archiver's ability to compress already-compressed data. The JPEG, PNG, and H.264 media are near- incompressible, providing an honest lower bound on what any format can achieve. The uncompressed TIFF and BMP images and the plain-text files are where formats can genuinely differentiate.

To make results byte-reproducible across machines, single-stream compressors (gzip, xz, Zstandard, and so on) and archivers that cannot store directory trees compress a *canonical tar* of each category: a deterministic archive with entries sorted by name, zeroed ownership, and fixed timestamps, so that its bytes—and therefore the resulting archive size—do not depend on the clone's filesystem metadata.

3.2 Format roster and level policy

We test 42 formats, organized into four acquisition tiers:

- **Tier A (native):** tools available from Homebrew or bundled with macOS.
- **Tier B (source builds):** vintage and experimental archivers compiled from upstream source, with small patches (committed in `tools/patches/`) where modern compilers or 64-bit ARM require them: ARC 5.21p, LHa, zoo 2.10, ARJ 3.10.22, HA 0.999, freeze 2.5, and paq8px.
- **Tier C (Wine):** Windows-only archivers run under Wine with Rosetta 2.
- **Tier D (DOSBox):** DOS-era archivers run under DOSBox-X, most notably PKZIP 2.04g (1993).

The **level policy** is central to the study's "practical" framing. Each tool is run at its **default** settings and at its **maximum** compression level—the one dial a casual user could reasonably change. We do *not* tune methods, dictionary sizes, filters, or thread counts beyond what the level switch implies. This produces some deliberate judgment calls, which we disclose rather than hide (Table 2).

Table 2: Level-policy judgment calls.

Format	Note
bzip2	Its default block size (`-9`) already is its maximum; default = max.
Brotli	The CLI's default quality is already the maximum (`-q 11`); default = max.
bzip3	Its only dial is block size; we treat default (16 MiB) and max (511 MiB) as the two levels.
Zstandard	Default level 3 vs. `--ultra -22` maximum.
DMG	We report UDZO (zlib, with a level dial) and ULMO (LZMA) as sibling format rows.
RAR	We report RAR5 and RAR4 (`-ma4`) as distinct formats, since the flag selects an archive-format version, not a tuning parameter.
zopfli	Reported as a separate "gzip, exhaustive" row: same `.gz` format, a different, far slower encoder.

3.3 Measurement protocol

For each (format, level, category) combination the harness stages the input, runs the create command, records the resulting archive size, then runs the extract command and verifies the result. Compression and decompression wall- clock times are the **median of three runs** for fast tools and a **single run** for very slow ones (zpaq at maximum, paq8px); peak resident memory is captured from `/usr/bin/time -l`. Timed runs are executed sequentially on an otherwise idle machine, kept awake with `caffeinate`. The exact hardware and every tool version are auto-recorded in `results/environment.md`.

Verification. Every archive is round-trip verified. For directory-mode formats we extract into a scratch directory, hash every file with SHA-256, and compare against the corpus manifest; content and relative path must match, while timestamps and permissions are ignored. For single-stream formats we confirm the decompressed tar is byte-identical to the canonical tar. Wherever possible

we verify with an *independent* extractor—for example, LHA archives created by our built `lha` are read back and cross-checked, ACE archives are read by the pure- Python `acefile` [19], and PKZIP's DOS output is read by modern Info-ZIP `unzip` [20]—so that a bug shared between a tool's compressor and decompressor cannot mask a corruption.

This verification is not a formality. On the image category, ARC 5.21p produced an archive that its *own* extractor unpacked while reporting a CRC failure and emitting a file 55 bytes longer than the original; the harness's SHA-256 check rejected it. Had we trusted the tool's exit status, a silent corruption would have passed as a successful result—a concrete argument for content-level verification over exit codes. Likewise, the 1980s `shar` shell-archive format succeeds on the text category but fails on every binary category, exactly as its text-only design predicts; we report these as honest failures rather than omitting the format.

Emulation policy. Formats produced under emulation (Tier C Wine, Tier D DOSBox) are flagged. Their archive **sizes are directly comparable** to native formats and are reported as first-class results; their **timings are excluded** because CPU emulation makes wall-clock time meaningless. These rows are drawn with hollow markers on the Pareto charts and shown with an em-dash in timing columns.

4 Results

The full, auto-generated ranking tables for every content category are in `results/tables/results.md`, and the complete machine-readable data is in `results/tables/results.csv`; the numbers below are drawn directly from those committed results. Ratio denotes archive size as a percentage of the original (lower is better).

4.0.1 Full corpus (55.07 MB)

Rank	Format	Level	Size	Ratio	Compress	Decompress	Notes
1	zpaq (journaling PAQ-family)	max	30.90 MB	56.11%	144.4	149.9	
2	tar.rz (rzip)	max	32.95 MB	59.82%	4.65	0.97	
3	tar.rz (rzip)	default	32.95 MB	59.83%	3.64	0.99	
4	7z (7-Zip, LZMA2)	max	33.02 MB	59.96%	4.43	0.43	
5	tar.xz (xz/LZMA2)	max	33.03 MB	59.98%	13.5	0.60	
6	7z (7-Zip, LZMA2)	default	33.03 MB	59.98%	4.13	0.42	
7	tar.lz (lzip)	max	33.21 MB	60.31%	16.7	2.28	

Rank	Format	Level	Size	Ratio	Compress	Decompress	Notes
8	tar.lrz (lrzip)	max	33.27 MB	60.41%	3.13	0.60	
9	tar.lrz (lrzip)	default	33.27 MB	60.42%	3.05	0.60	
10	tar.zst (Zstandard)	max	33.78 MB	61.33%	6.82	0.05	
11	tar.br (Brotli)	default	35.80 MB	65.00%	84.0	0.23	
12	tar.bz3 (bzip3)	max	36.74 MB	66.72%	7.46	6.87	
13	tar.bz3 (bzip3)	default	38.24 MB	69.43%	7.16	5.79	
14	zpaq (journaling PAQ-family)	default	39.90 MB	72.46%	0.72	1.84	
15	tar.xz (xz/LZMA2)	default	40.21 MB	73.02%	6.73	0.35	
16	tar.lz (lzip)	default	40.45 MB	73.46%	15.0	2.73	
17	RAR 4 (legacy format)	max	42.38 MB	76.96%	0.90	0.41	
18	RAR 4 (legacy format)	default	42.43 MB	77.05%	0.81	0.40	
19	tar.bz2 (bzip2)	default	42.92 MB	77.93%	3.84	1.76	
20	RAR 5 (WinRAR/rar)	max	43.01 MB	78.09%	1.02	0.20	
21	RAR 5 (WinRAR/rar)	default	43.08 MB	78.22%	0.81	0.21	
22	DMG (ULMO, lzma-compressed image)	default	43.97 MB	79.83%	11.1	0.84	
23	tar.gz (zopfli, exhaustive deflate)	default	44.56 MB	80.91%	59.7	0.13	

Rank	Format	Level	Size	Ratio	Compress	Decompress	Notes
24	LHA/LZH (1988, lh5)	max	44.82 MB	81.39%	3.87	0.89	
25	ZIP (PKZIP 2.04g, 1993) (emu)	max	44.97 MB	81.66%	—	—	emulated (size only)
26	WIM (wimlib, LZX)	max	45.04 MB	81.78%	0.40	0.14	
27	WIM (wimlib, LZX)	default	45.06 MB	81.83%	0.29	0.15	
28	ZIP (PKZIP 2.04g, 1993) (emu)	default	45.07 MB	81.84%	—	—	emulated (size only)
29	ZIP (Info-ZIP)	max	45.09 MB	81.87%	1.50	0.32	
30	dar (Disk ARchive)	default	45.11 MB	81.92%	1.73	0.30	
31	xar (macOS eXtensible ARchiver)	default	45.11 MB	81.92%	1.53	0.12	
32	ZIP (Info-ZIP)	default	45.12 MB	81.92%	1.35	0.33	
33	tar.gz (gzip)	max	45.12 MB	81.92%	1.50	0.10	
34	tar.gz (gzip)	default	45.14 MB	81.97%	1.34	0.09	
35	ARJ (1991)	max	45.20 MB	82.07%	1.09	0.63	
36	ARJ (1991)	default	45.20 MB	82.08%	1.05	0.65	
37	SquashFS (mksquashfs)	default	45.25 MB	82.16%	0.20	0.04	

Rank	Format	Level	Size	Ratio	Compress	Decompress	Notes
38	tar.zst (Zstandard)	default	45.40 MB	82.44%	0.08	0.05	
39	Apple Archive (.aar, lzfse)	default	45.46 MB	82.54%	0.09	0.06	
40	LHA/LZH (1988, lh5)	default	45.53 MB	82.67%	2.29	1.04	
41	zoo (1986)	max	45.62 MB	82.83%	2.83	1.83	
42	CAB (gcab, MSZIP)	default	45.68 MB	82.95%	0.96	0.16	
43	HA (1993, PPM-family)	default	45.74 MB	83.06%	4.22	3.77	
44	freeze (1993)	default	45.75 MB	83.07%	4.12	3.05	
45	DMG (UDZO, zlib-compressed image)	max	46.09 MB	83.69%	6.07	0.32	
46	tar.lzo (lzop)	max	46.50 MB	84.44%	3.75	0.07	
47	tar.lz4 (LZ4)	max	46.51 MB	84.45%	0.39	0.05	
48	DMG (UDZO, zlib-compressed image)	default	46.79 MB	84.96%	6.08	0.35	
49	tar.lz4 (LZ4)	default	49.50 MB	89.88%	0.03	0.03	
50	tar.lzo (lzop)	default	49.53 MB	89.94%	0.09	0.08	
51	cpio (appendix: store-only)	default	55.07 MB	100.00%	0.26	0.06	

Rank	Format	Level	Size	Ratio	Compress	Decompress	Notes
52	tar (uncompressed, baseline)	default	55.09 MB	100.03%	0.06	0.05	
53	ar (appendix: unix archiver, store-only)	default	55.10 MB	100.05%	0.17	0.08	
54	pax (appendix: POSIX archiver)	default	55.10 MB	100.05%	0.10	0.10	
55	zoo (1986)	default	55.10 MB	100.05%	2.65	0.18	
56	ISO 9660 (mkisofs, store-only)	default	55.46 MB	100.70%	0.03	0.03	
57	tar.Z (compress, LZW 1984)	default	58.36 MB	105.97%	1.05	0.47	
58	ARC (SEA, 1985)	default	62.29 MB	113.11%	1.33	0.26	

4.0.2 Text (11.02 MB)

Rank	Format	Level	Size	Ratio	Compress	Decompress	Notes
1	paq8px (context mixing, state of the art)	max	1.96 MB	17.76%	18.8m	18.9m	
2	paq8px (context mixing, state of the art)	default	1.99 MB	18.04%	15.2m	15.2m	

Rank	Format	Level	Size	Ratio	Compress	Decompress	Notes
3	zpaq (journaling PAQ-family)	max	2.19 MB	19.85%	26.8	27.9	
4	tar.bz3 (bzip3)	default	2.42 MB	21.96%	1.17	0.65	
5	tar.bz3 (bzip3)	max	2.42 MB	21.96%	1.54	0.84	
6	tar.bz2 (bzip2)	default	2.85 MB	25.81%	0.58	0.21	
7	tar.rz (rzip)	default	2.91 MB	26.42%	0.73	0.30	
8	tar.rz (rzip)	max	2.92 MB	26.50%	1.06	0.32	
9	tar.lz (lzip)	max	2.96 MB	26.89%	3.84	0.22	
10	tar.xz (xz/LZMA2)	max	2.97 MB	26.92%	3.50	0.13	
11	tar.xz (xz/LZMA2)	default	2.97 MB	26.92%	3.23	0.12	
12	7z (7-Zip, LZMA2)	max	2.97 MB	26.93%	2.00	0.09	
13	tar.lz (lzip)	default	2.97 MB	26.98%	3.59	0.22	
14	7z (7-Zip, LZMA2)	default	2.98 MB	27.03%	1.96	0.09	
15	tar.br (Brotli)	default	2.98 MB	27.05%	13.5	0.04	
16	tar.lrz (lrzip)	default	3.00 MB	27.24%	2.45	0.23	
17	tar.lrz (lrzip)	max	3.00 MB	27.24%	2.66	0.23	
18	tar.zst (Zstandard)	max	3.03 MB	27.48%	3.03	0.02	
19	RAR 4 (legacy format)	max	3.25 MB	29.45%	0.27	0.06	
20	RAR 5 (WinRAR/rar)	max	3.25 MB	29.48%	0.27	0.04	

Rank	Format	Level	Size	Ratio	Compress	Decompress	Notes
21	DMG (ULMO, lzma-compressed image)	default	3.28 MB	29.75%	7.07	0.36	
22	RAR 4 (legacy format)	default	3.30 MB	29.92%	0.18	0.06	
23	RAR 5 (WinRAR/rar)	default	3.30 MB	29.95%	0.17	0.05	
24	tar.gz (zopfli, exhaustive deflate)	default	3.66 MB	33.20%	14.6	0.03	
25	LHA/LZH (1988, lh5)	max	3.73 MB	33.84%	1.12	0.14	
26	tar.zst (Zstandard)	default	3.80 MB	34.48%	0.04	0.02	
27	ZIP (Info-ZIP)	max	3.85 MB	34.92%	0.54	0.07	
28	ZIP (PKZIP 2.04g, 1993) (emu)	max	3.85 MB	34.95%	—	—	emulated (size only)
29	tar.gz (gzip)	max	3.85 MB	34.96%	0.53	0.04	
30	xar (macOS eXtensible ARchiver)	default	3.85 MB	34.96%	0.56	0.04	
31	dar (Disk ARchive)	default	3.85 MB	34.96%	0.59	0.05	
32	HA (1993, PPM-family)	default	3.86 MB	34.99%	0.81	0.35	

Rank	Format	Level	Size	Ratio	Compress	Decompress	Notes
33	ZIP (Info-ZIP)	default	3.87 MB	35.14%	0.41	0.09	
34	tar.gz (gzip)	default	3.88 MB	35.18%	0.40	0.04	
35	ZIP (PKZIP 2.04g, 1993) (emu)	default	3.92 MB	35.52%	—	—	emulated (size only)
36	Apple Archive (.aar, lzfse)	default	3.93 MB	35.66%	0.06	0.04	
37	ARJ (1991)	max	3.94 MB	35.75%	0.52	0.15	
38	ARJ (1991)	default	3.94 MB	35.77%	0.47	0.16	
39	SquashFS (mk-squashfs)	default	3.96 MB	35.89%	0.08	0.03	
40	DMG (UDZO, zlib-compressed image)	max	4.01 MB	36.35%	6.07	0.22	
41	WIM (wimlib, LZX)	max	4.01 MB	36.38%	0.20	0.05	
42	WIM (wimlib, LZX)	default	4.03 MB	36.53%	0.13	0.05	
43	zpaq (journal-ing PAQ-family)	default	4.03 MB	36.59%	0.32	1.18	
44	LHA/LZH (1988, lh5)	default	4.19 MB	38.02%	0.45	0.16	
45	zoo (1986)	max	4.21 MB	38.15%	0.73	0.23	
46	freeze (1993)	default	4.23 MB	38.35%	0.58	0.22	

Rank	Format	Level	Size	Ratio	Compress	Decompress	Notes
47	CAB (gcab, MSZIP)	default	4.25 MB	38.55%	0.32	0.05	
48	tar.Z (compress, LZW 1984)	default	4.30 MB	39.03%	0.17	0.09	
49	tar.lz4 (LZ4)	max	4.44 MB	40.26%	0.28	0.03	
50	tar.lzo (lzop)	max	4.50 MB	40.80%	1.46	0.03	
51	DMG (UDZO, zlib-compressed image)	default	4.79 MB	43.44%	4.08	0.23	
52	zoo (1986)	default	5.17 MB	46.93%	0.25	0.09	
53	ARC (SEA, 1985)	default	5.53 MB	50.12%	0.20	0.12	
54	tar.lzo (lzop)	default	6.23 MB	56.53%	0.05	0.05	
55	tar.lz4 (LZ4)	default	6.40 MB	58.09%	0.02	0.02	
56	cpio (appendix: store-only)	default	11.02 MB	100.01%	0.07	0.02	
57	tar (uncompressed, baseline)	default	11.03 MB	100.04%	0.03	0.03	
58	pax (appendix: POSIX archiver)	default	11.04 MB	100.10%	0.04	0.04	
59	ar (appendix: unix archiver, store-only)	default	11.04 MB	100.10%	0.07	0.04	

Rank	Format	Level	Size	Ratio	Compress	Decompress	Notes
60	shar (appendix: shell archive)	default	11.29 MB	102.40%	0.13	0.18	
61	ISO 9660 (mkisofs, store-only)	default	11.38 MB	103.24%	0.03	0.01	

4.0.3 Best format per content category

Category	Smallest format	Ratio	Practical winner (mainstream)
Text	paq8px (context mixing, state of the art) (max)	17.76%	tar.xz (xz/LZMA2) — 26.92%
Documents	zpaq (journaling PAQ-family) (max)	59.31%	tar.zst (Zstandard) — 59.72%
Images	zpaq (journaling PAQ-family) (max)	67.37%	RAR 5 (WinRAR/rar) — 74.25%
Video	zpaq (journaling PAQ-family) (max)	98.50%	tar.xz (xz/LZMA2) — 98.62%
Full corpus	zpaq (journaling PAQ-family) (max)	56.11%	7z (7-Zip, LZMA2) — 59.96%

4.0.4 Attempted but not completed

Format	Level	Dataset	Outcome	Reason
ARC (SEA, 1985)	default	images	failed	extract exited 1:
FreeArc 0.666 (2007, Wine)	default	full	timeout	create timed out after 120s
FreeArc 0.666 (2007, Wine)	max	full	timeout	create timed out after 120s
FreeArc 0.666 (2007, Wine)	default	text	timeout	create timed out after 120s
FreeArc 0.666 (2007, Wine)	max	text	timeout	create timed out after 120s

Format	Level	Dataset	Outcome	Reason
shar (appendix: shell archive)	default	documents	failed	create exited 1: sed: RE error: illegal byte sequence
shar (appendix: shell archive)	default	full	failed	create exited 1: sed: RE error: illegal byte sequence
shar (appendix: shell archive)	default	images	failed	create exited 1: sed: RE error: illegal byte sequence
shar (appendix: shell archive)	default	video	failed	create exited 134: Assertion failed: (advance > 0), function substitute, file pr
UHARC 0.6b (2005, Wine)	default	full	timeout	create timed out after 120s
UHARC 0.6b (2005, Wine)	max	full	timeout	create timed out after 120s
UHARC 0.6b (2005, Wine)	default	text	timeout	create timed out after 120s
UHARC 0.6b (2005, Wine)	max	text	timeout	create timed out after 120s

4.1 The size/speed frontier

Figures 1 and 2 plot every native format on the size/time plane for the full corpus and for text. The frontier is steep and then flat: moving from gzip to xz or 7-Zip buys a large ratio improvement for a modest time cost, but pushing further—into zpaq's maximum mode or paq8px—buys a few more percentage points at one to three *orders of magnitude* more time. For a casual user, the knee of this curve, occupied by 7-Zip, xz, and Zstandard, is where the practical choice lies.

5 Discussion

What should a casual user actually use? Three answers, by need:

- *For universal compatibility*—a file anyone can open with no extra software—plain **ZIP** remains the correct choice. On the full mixed corpus it reached 82% of the original size; it is not the smallest, but every operating system has opened it natively for thirty years.
- *For the best mainstream size/speed balance*, **7-Zip** (`.7z`) and **Zstandard** (`.tar.zst`) dominate the knee of the frontier. On the full corpus 7-Zip reached **60% in about four seconds**—a quarter smaller than ZIP for no perceptible extra effort—and Zstandard reached 61%. On the office- document category the gap is starker still: ZIP left the already-compressed `.docx`/`.xlsx`

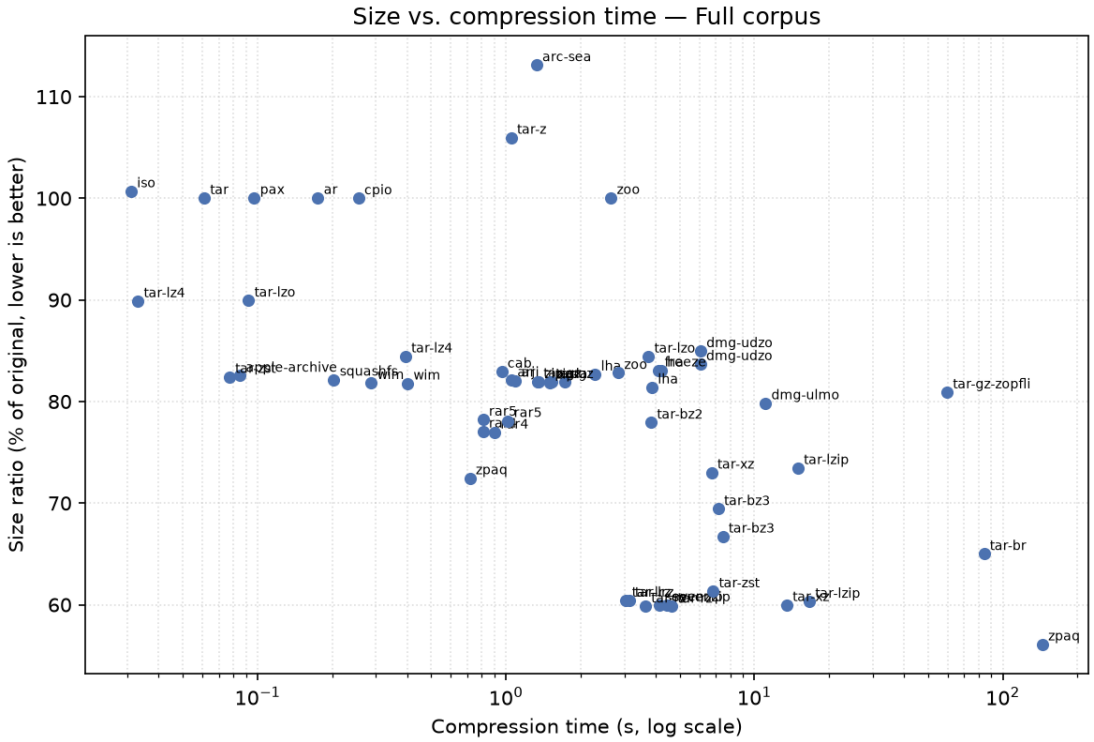

Figure 1: Size versus compression time on the full corpus. Lower and to the left is better; the x-axis is logarithmic. Emulated formats are omitted (their timings are not comparable).

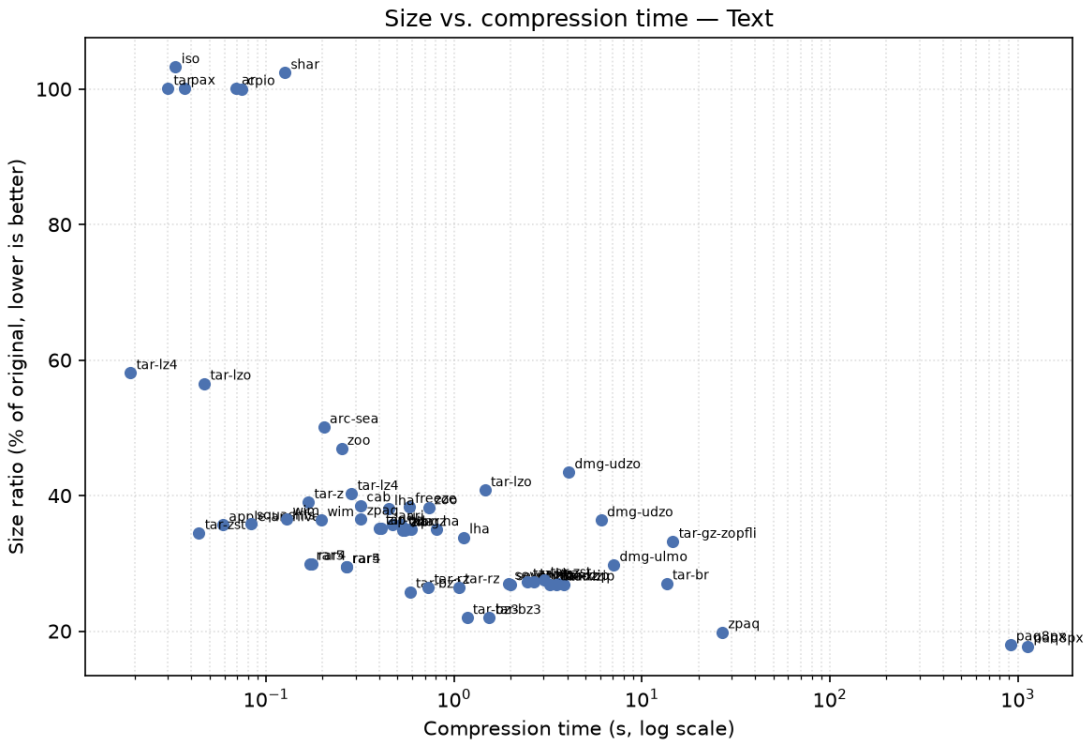

Figure 2: Size versus compression time on the text category, where formats differ most.

files essentially untouched at 99%, while 7-Zip recompressed them to **60%** by working across the archive members.

- *For the absolute smallest file*, when time is no object, the journaling context-mixing archiver **zpaq** reached **56%** at its maximum setting—the smallest of any format on the full corpus—but took roughly 150 seconds to compress and again to decompress, against 7-Zip's four. The even more extreme context-mixing **paq8px** goes further still, compressing the text category to **17.8%**—the smallest result any format achieved on any category—but at a throughput near 12 KB/s: roughly fifteen to nineteen minutes to compress the 11 MB of text, and as long again to decompress, with an archive only paq8px itself can open.

Diminishing returns are dramatic. On text, the best mainstream tools (bzip2 at 25.8%, 7-Zip and xz at about 27%) sit within a few percentage points of the exotic maximum—zpaq's 19.9% and paq8px's 17.8%—while costing a fraction of a second rather than the minutes those extreme tools demand. Plain ZIP reached 35%—and, strikingly, the *1993* PKZIP 2.04g binary we ran under DOS emulation produced a nearly identical 35.5%, a reminder that the ZIP format a casual user gets today is essentially the one from three decades ago. This is the single most important practical takeaway: beyond the mainstream tools, users pay large time and compatibility costs for small size gains.

Already-compressed data resists everyone. On the video category every format—old or new—clustered between 98% and 100% of the original size, because H.264 is already compressed; several formats even *expanded* the data slightly through container overhead. The corollary for users is that "zipping" a folder of videos saves essentially nothing; the value of an archive there is bundling and convenience, not size. Photographs behaved similarly, though the uncompressed TIFF and

BMP images in our set gave the stronger compressors (RAR4 at 70%, 7-Zip at 76%) some room to work.

Format longevity cuts both ways. Remarkably, every vintage *format* in our set from 1985 onward can still be both created and read on a 2026 machine, once its tool is rebuilt from source—ARC, zoo, LHA, ARJ, HA, and freeze all round- trip cleanly. A 1993 PKZIP binary, run under DOS emulation, still produces a ZIP that a modern `unzip` opens. Yet the *Windows-only* archivers of the 2000s fared worse: FreeArc and UHARC, run under Wine with Rosetta 2 on Apple Silicon, consistently deadlocked in their compression cores under double binary translation, and ACE creation is available only through a discontinued graphical installer. Paradoxically, a 40-year-old open DOS format proved more durable than a 20-year-old proprietary Windows one—an argument, for archival purposes, in favor of simple, open, well-documented formats.

6 Limitations

This is a single-machine study; absolute timings will differ on other hardware, though relative orderings should be stable. Multithreaded tools (7-Zip, Zstandard, some others) are run at their default thread behavior, which favors them on wall-clock time; we disclose this rather than force single-threading, because default behavior is what a user experiences. Timings for emulated tools are excluded entirely. The corpus is 55 MB—large enough to be realistic, small enough to commit and to keep paq8px tractable—but ratios can shift with scale, particularly for long-range and dictionary-based methods. Our level policy intentionally forgoes method tuning, so specialist configurations that an expert might use (for example PPMd for text in 7-Zip) are out of scope by design. We did not evaluate encryption, archive splitting, solid-archive tuning, or recovery records.

7 Reproducibility

The entire study is reproducible from the repository with a single command, `./run_all.sh`, which verifies the corpus, runs the benchmark, regenerates the tables and figures, and rebuilds this paper. The corpus and all raw per-run JSON results are committed, so the tables and figures can be regenerated—and every claim in this paper checked—without re-running the multi-hour benchmark. The exact environment is recorded in `results/environment.md`; tool provenance, including download URLs and SHA-256 checksums for every non-package binary, is in `tools/README.md`.

8 Conclusion

Across 42 formats and four decades, the practical advice for a casual user is simple and perhaps reassuring: **ZIP for sharing, 7-Zip or Zstandard for size.** The exotic tail of formats is fascinating—and we have shown that most of it still works, some of it heroically—but it offers ordinary users little that the mainstream does not, at costs in time and compatibility they should not pay. The enduring lesson is about openness and durability: the formats that survived four decades did so because they were simple, open, and well documented, and those are the properties worth valuing in a format meant to outlast the software that made it.

References

[1] L. P. Deutsch, "DEFLATE compressed data format specification version 1.3," Internet Engineering Task Force, RFC 1951, 1996. doi: 10.17487/RFC1951.

[2] J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," *IEEE Transactions on Information Theory*, vol. 23, no. 3, pp. 337–343, 1977, doi: 10.1109/TIT.1977.1055714.

[3] Y. Collet and M. S. Kucherawy, "Zstandard compression and the 'application/zstd' media type," Internet Engineering Task Force, RFC 8878, 2021. doi: 10.17487/RFC8878.

[4] J. Alakuijala and Z. Szabadka, "Brotli compressed data format," Internet Engineering Task Force, RFC 7932, 2016. doi: 10.17487/RFC7932.

[5] E. Nemerson, "Squash compression benchmark." https://quixdb.github.io/squash-benchmark/, 2015.

[6] P. Skibinski, "Lzbench: In-memory benchmark of open-source LZ77/LZSS/LZMA compressors." https://github.com/inikep/lzbench, 2016.

[7] M. V. Mahoney, "Large text compression benchmark." http://mattmahoney.net/dc/text.html, 2006.

[8] S. Deorowicz, "The silesia compression corpus." http://sun.aei.polsl.pl/~sdeor/index.php?page=silesia, 2003.

[9] T. A. Welch, "A technique for high-performance data compression," *Computer*, vol. 17, no. 6, pp. 8–19, 1984, doi: 10.1109/MC.1984.1659158.

[10] System Enhancement Associates and H. C. Yen, "ARC – SEA archive utility." https://github.com/hyc/arc.

[11] paq8px contributors, "paq8px." https://github.com/hxim/paq8px.

[12] M. V. Mahoney, "Fast text compression with neural networks," in *Proceedings of the thirteenth international florida artificial intelligence research society conference (FLAIRS)*, AAAI Press, 2000, pp. 230–234.

[13] L. P. Deutsch, "GZIP file format specification version 4.3," Internet Engineering Task Force, RFC 1952, 1996. doi: 10.17487/RFC1952.

[14] PKWARE Inc.,".ZIP file format specification (APPNOTE.TXT)." https://pkware.cachefly.net/webdocs/casestudies/APPNOTE.TXT, 2020.

[15] M. Burrows and D. J. Wheeler, "A block-sorting lossless data compression algorithm," Digital Systems Research Center, SRC Research Report 124, 1994. Available: https://www.hpl.hp.com/techreports/Compaq-DEC/SRC-RR-124.pdf

[16] Project Gutenberg, "Project gutenberg." https://www.gutenberg.org/.

[17] U.S. Geological Survey, "USGS earthquake catalog." https://earthquake.usgs.gov/earthquakes/feed/.

[18] National Aeronautics and Space Administration, "NASA image and video library." https://images.nasa.gov/.

[19] D. Roethlisberger, "Acefile: Read/test/extract ACE 1.0 and 2.0 archives in pure python." https://www.roe.ch/acefile.

[20] Info-ZIP, "Info-ZIP." https://infozip.sourceforge.net/.